

UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

PLATFORMĂ DE VIZUALIZARE ȘI EDITARE DE GRAFURI 3D FOLOSIND WEBGL ȘI COMPILATOR DE EXPRESII MATEMATICE ÎN SHADERE

Absolvent

Păcurariu Răzvan Mihai

Coordonator științific

Lect. Dr. Carmen Elena Chiriță

București, iunie - iulie 2026

Rezumat

Această licență implementează o platformă digitală destinată creării și editării scenelor formate din grafuri 3D. Lucrarea are ca scop crearea unui sistem eficient și a unei interfețe intuitive care îi permite utilizatorului să lucreze cu grafuri explicite, implicite și parametrice. Pe lângă asta, un strat social este implementat, în care un utilizator poate să distribuie scene create de el și să reacționeze și să comenteze la ele. Ei mai pot să deschidă și să editeze scene create de alți utilizatori.

Aplicația este concepută să răspundă la nevoile a două categorii distincte de utilizatori: utilizatori normali, care au acces la editorul de scene și la elementele sociale; și adminii, care pot analiza raporturi create de utilizatori pentru anumite postări sau comentarii, și pot să le șteargă sau să baneze utilizatorul care le-a creat.

Platforma a fost dezvoltată cu o arhitectură full-stack, folosind **React** pe frontend ca un SPA (Single Page Application); iar pe backend folosind **.NET Core**, susținută de o bază de date relațională. Au fost implementate funcționalități pentru interfața de grafare, feed-ul de social media și dashboard-uri pentru operații administrative.

Rezultatul obținut este o aplicație web funcțională, modulară și extensibilă. Contribuția principală a acestei lucrări a fost realizarea interfeței și a sistemului de grafare 3D, împreună cu îmbinarea ei cu elemente sociale.

Abstract

This thesis implements a digital platform for creating and editing scenes formed from 3D graphs. The project has the goal of creating an efficient system and intuitive interface which allows the user to work with explicit, implicit and parametric graphs. Alongside this, a social layer is implemented, where a user can share scenes created by them, and react and comment on them. They can also open and edit scenes created by other users.

The application was conceptualized to respond to the needs of two distinct categories of users: regular users, that have access to the scene editor and to the social elements; and admins, that can analyze reports created by users for certain posts and comments, and can delete them or ban the user that made them.

The platform was developed using a full-stack architecture, using **React** for the frontend as a SPA (Single Page Application); and .NET Core for the backend, supported by a relation database. Implemented features include the graphing interface, the social media feed, and dashboards for administrative operations.

The final result is a functional, modular and extensible web app. The main contribution of this work was the creation of the graphical interface and system, alongside combining it with social media elements.

Cuprins

1 Introducere **4**
1.1 Prezentarea generală a temei 4
1.2 Scopul și motivația alegerii temei 4
1.3 Contribuția proprie în realizarea lucrări 5
1.4 Structura lucrării 5

2 Preliminarii **6**
2.1 Tehnologii folosite 6
2.2 Backend 7
2.3 Frontend 8
2.4 Grafare 8

3 Proiectarea și implementarea proiectului **9**
3.1 Structura aplicației 9
3.1.1 Backend 9
3.1.2 Frontend 10
3.2 Baza de date 10
3.3 Interfața de grafare 12
3.4 Roluri 12
3.4.1 Utilizator neautentificat 13
3.4.2 Utilizator normal 13
3.4.3 Admin 13

4 Ghidul de utilizare al aplicației **14**
4.1 Utilizator neautentificat 14
4.2 Utilizator autentificat 15
4.3 Admin 17

5 Concluzii **20**

Bibliografie **22**

Capitolul 1

Introducere

1.1 Prezentarea generală a temei

În prezent, uneltele de vizualizare matematică există adesea ca editoare izolate, dar rareori oferă un model de comunitate pentru distribuirea, discutarea, moderarea și refolosirea construcțiilor cu grafuri, în special folosind toate aceste elemente. Unele exemple sunt *Desmos* și *GeoGebra*, care oferă capabilități de salvare și distribuire a scenelor create, dar nu de a comenta pe ele.

Platformele sociale, pe de altă parte, de obicei tratează elementele vizuale ca artefacte statice, care nu pot fi editate după postarea lor; iar în cazul în care pot, nu într-o manieră care se pretează la un acces mai detaliat al structurii elementului vizual, de obicei utilizatorul putând să editeze doar peste imaginea finală.

1.2 Scopul și motivația alegerii temei

Scopul lucrării este să adreseze această discrepanță prin tratarea ale unei scene matematice 3D atât ca un obiect editabil, cât și ca un obiect social. Pentru asta, o aplicație web completă și funcțională a fost realizată, care răspunde la cerințele problemei și la cele uzuale ale unei aplicații de grafare și ale unei platforme sociale, oferind o experiență optimizată pentru utilizator și un instrument eficient pentru crearea scenelor 3D.

Motivația alegerii temei derivă din mai multe puncte. O dată, este din interesele personale de a învăța și a lucra cu tehnologi pentru folosirea elementelor grafice pe GPU pe web, de a crea o platformă socială care are ca element unic principal ceva pe care m-aș vedea eu însuși folosind; dar și din golul identificat pe piață al acestui tip de platforme sociale.

Experiențe relevante sunt lucrările făcute cu aceste tehnologii, React și .NET Core, împreună cu cunoștințele și interesul în grafică.

1.3 Contribuția proprie în realizarea lucrări

Contribuția proprie în cadrul acestei lucrări a fost realizarea completă a aplicației web, identificarea rolurilor și a drepturilor pentru fiecare rol, proiectarea și implementarea arhitecturii aplicației și a bazei de date, alegerea tehnologiilor potrivite și, în final, implementarea eficientă a sistemului de grafare.

Aceste elemente au fost implementate ținând cont de o experiență optimizată pentru utilizator, oferind-ui o interfață vizuală coerentă și responsive.

1.4 Structura lucrării

Capitolul 1 - Introducere

Capitolul oferă o prezentare generală a domeniului tehnologic în care se încadrează tema propusă. Se descrie contextul actual a aplicațiilor de grafare și a platformelor sociale, și justificarea alegerii subiectului. De asemenea, se prezintă contribuțiile originale aduse de realizarea proiectului, și o prezentare a structurii generale a lucrării.

Capitolul 2 - Preliminarii

Capitolul oferă conceptele și tehnologiile fundamentale care au fost folosite pentru dezvoltarea aplicației, precum arhitectura și componenta de grafare.

Capitolul 3 - Proiectarea și implementarea aplicației

Această secțiune prezintă în detaliu metodele prin care componentele platformei au fost realizate, împreună cu proiectarea arhitecturii și detalii de implementare, și motivația pentru ele.

Capitolul 4 - Ghidul de utilizare al aplicației

Capitolul detaliază interacțiunea cu platforma descrisă anterior, din punctul de vedere al fiecărui rol definit. Sunt prezentate diferitele funcționalități disponibile și fluxurile logice ale experienței, împreună cu capturi de ecran ale aplicației.

Capitolul 5 - Concluzii

În acest capitol se prezintă rezultatele obținute și gradul de atingere a obiectivelor propuse inițial. Se descriu și direcțiile viitoare pentru proiect, precum îmbunătățiri care s-ar putea realiza sau funcționalități noi.

Capitolul 2

Preliminarii

2.1 Tehnologii folosite

Pentru dezvoltarea aplicației, a fost folosită o arhitectură full-stack, care permite o separare clară între backend, logica aplicației și frontend, interfața cu utilizatorul. Tehnologiile au fost alese din cauza experienței anterioare și a capacitatăților acestora de a oferi performanță și modularitate.

Aplicația a fost dezvoltată folosind **Visual Studio 2022** ca IDE, care oferă suport extins pentru .NET Core și .NET 8 [1], baza backend-ului, dar și pentru gestionarea bazelor de date SQL și a pachetelor (prin NuGet), folosite pentru Entity Framework [2] și pentru Identity.

Pentru frontend, pe lângă Visual Studio 2022, s-a folosit și **Visual Studio Code**, care oferă suport pentru React [6] și TypeScript [12]. Oferă și integrare cu Git, având și un terminal integral care a fost folosit la rularea rapidă a comenzilor de build și rularea componentei frontend, conducând la o iterare rapidă și ușoară a interfeței vizuale.

Vite [13], folosit pentru dezvoltarea frontend-ului, este o unealtă de build care permite iterarea rapidă a interfeței, întrucât nu trebuie reconstruită toată aplicația când o singură componentă este modificată.

Server-ul este configurat ca aplicație de host. Proiectul ASP.NET se raportează la clientul React printr-o configurație proxy SPA. Vite servește client-ul peste HTTPS și face proxy-urile /api și /uploads.

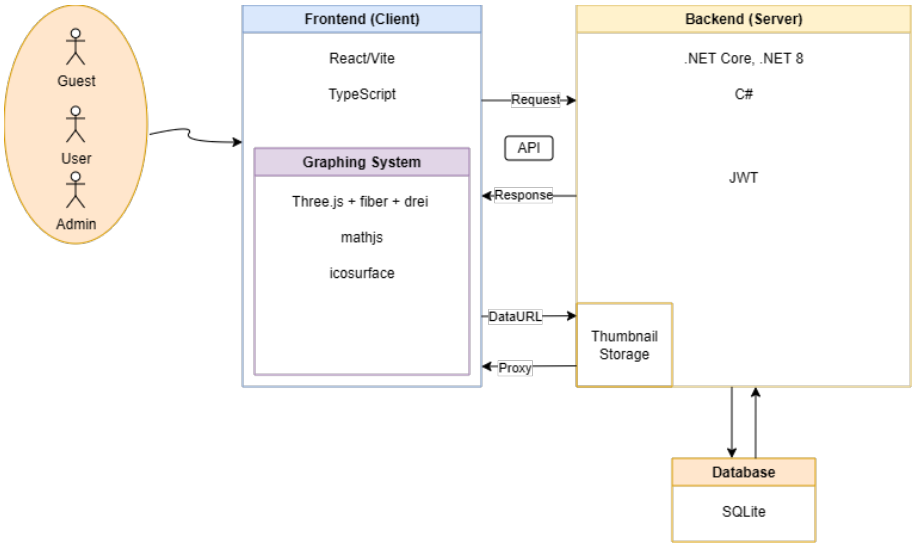


Figura 2.1: Arhitectura aplicației.

2.2 Backend

În partea de backend, s-au folosit platformele .NET Core și .NET 8, dezvoltate în limbajul C#, mediu eficient pentru dezvoltarea de API-uri REST. Pentru baza de date, s-a folosit o bază de date SQLite [7]. Integrarea entităților din această bază de date a fost realizată prin folosirea a Microsoft Entity Framework Core, care oferă maparea obiect-relațională și scrierea de interogări LINQ (Language Integrated Query). Pentru asigurarea consistenței între versiunile bazei de date, cât timp se lucra la arhitectura ei, s-a folosit sistemul de migrări pus la dispoziție de Entity Framework.

Pentru autentificare și securitate, s-a folosit JWT (JSON Web Token) [4], mecanism eficient pentru autentificarea și autorizarea utilizatorilor. Un JWT este un token semnat care conține cereri (claims). În acest proiect, ele includ id-ul, email-ul, numele, subiectul, id-ul de token și claim-urile de rol. Are și un emitent, audiență, timp not-before și timp de expirare. Serverul semnează token-ul cu o cheie simetrică HMAC SHA-256.

Pentru experiența utilizatorului, capturi ale scenelor sunt folosite pentru scene pe frontend. Ele sunt generate de client, trimise către server printr-un dataURL, și stocate într-un folder al server-ului. Client-ul the React/Vite le obține înapoi prin proxy-ul /uploads.

Structura aplicației respectă principiile arhitecturii de tip microservicii, fiind o separare clară între controllere, servicii, și stratul de acces a datelor. S-a utilizat Dependency Injection pentru injectarea serviciilor în controllere, precum cele deja existente, ca IdentityDbContext pentru folosirea bazei de date, dar și cele create de mine, precum JwtTokenService, un wrapper pentru folosirea mai ușoară a JWT-urilor.

Toate endpoint-urile ale aplicației au fost expuse prin intermediul lui Swagger [8], pentru vizualizarea și testarea cu ușurință a API-urilor.

Git a fost folosit pentru gestionarea codului sursă și lucrarea în ramuri separate pentru fiecare funcționalitate nouă.

2.3 Frontend

În partea de frontend, s-au folosit biblioteca React și limbajul TypeScript pentru construirea interfeței. S-au creat componente scalabile, modulare și reutilizabile, cu accent pe un design responsive și optimizat, în special pentru versiunea desktop. Componentele au fost adnotate cu tipuri clare pentru a minimiza erorile de utilizare.

Pentru interfața de grafare, s-au folosit mai multe pachete: **Three.js** [9], librărie modernă pentru folosirea accelerări grafice în browser, împreună cu **Three.js/fiber** [11], care oferă o interfață de folosire a acestei librării cu React. **Three.js/drei** [10] oferă un controler pentru cameră, iar **mathjs** [5] oferă engine-ul de parsare și compilare a expresiilor matematice. În final, **isosurface** [3] oferă capabilitățile pentru randarea eficientă a suprafețelor grafurilor.

2.4 Grafare

Randarea suprafețelor explicite și parametrice este relativ ușoară, întrucât pentru amândouă se poate itera peste un grid de puncte pentru a construi suprafața.

Pentru suprafețe implicite, se folosește algoritmul de *Marching Cubes* din librăria isosurface. El tratează un spațiu 3D ca un grid de cuburi mici. Fiecare colț a unui cub are o valoare scalară, calculată prin evaluarea lui $F(x, y, z)$, formula pentru suprafața implicită. Pentru fiecare cub, algoritmul verifică dacă fiecare dintre cele 8 colțuri este înăuntru sau în afara suprafeței, comparând valorile scalare cu zero. Asta creează $2^8 = 256$ cazuri posibile, pentru care algoritmul folosește un tabel precalculat care descrie cum să genereze triunghiurile pentru suprafața în acel cub.

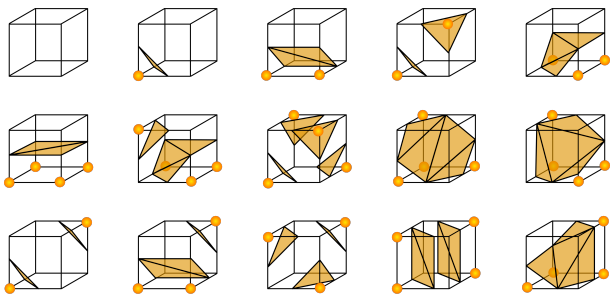


Figura 2.2: Exemple de configurații ale colțurilor cu triunghiurile generate. (Sursă: Wikipedia)

Capitolul 3

Proiectarea și implementarea proiectului

Această secțiune prezintă dezvoltarea aplicativă a temei abordate, explicând în detaliu componentele principale ale aplicației web. Se începe de la structurarea arhitecturii proiectului, arhitectura bazei de date, și apoi dezvoltarea funcționalităților specifice proiectului.

3.1 Structura aplicației

Aplicația a fost organizată conform unei arhitecturi multi-layered, o structurare care permite claritate, scalabilitate și modularitate. Componentele pot fi dezvoltate cu ușurință și în paralel, oferind un grad ridicat de mentenanță.

3.1.1 Backend

Backend-ul, dezvoltat cu **.NET Core** și **NET 8**, este împărțit în:

Controllers: Punctul de intrare pentru cererile HTTP de la client. Controller-urile sunt asociate unor anumite categorii din proiect. De exemplu: AuthController este responsabil pentru logica de înregistrare, autentificare, delogare; ScenesController este responsabil pentru logica de salvare, updatare și obținere de scene; PostsController este responsabil pentru logica de CRUD a postărilor pe partea de rețea socială a aplicației, împreună cu like-urile și comentariile asociate lor.

Services: Aici se află logică folositoare pentru anumite funcționalități des utilizate, precum JwtTokenService care este responsabil pentru crearea de JWT-uri, și SceneThumbnailStorageService care este responsabil pentru parsarea și salvarea imaginilor de scenă pe server.

Models: Definește entitățile aflate în baza de date; împreună cu ApplicationDbContext, care definește relațiile dintre aceste entități și creează, folosind Entity Framework Core, ORM-ul (Object-Relational Mapping), permițând lucrul cu entități sub formă de obiecte C#.

3.1.2 Frontend

Frontend-ul a fost dezvoltat folosind **React** și **TypeScript**.

El a fost modularizat pentru iterarea și extinderea ușoară a aplicației.

Punctul de intrare este App.tsx, un SPA (Single Page Application), care conține Navbar-ul, componenta de autorizare (care conține și meniul) ca să fie activă tot timpul, și un React Router, care definește rute publice și protejate.

Funcționalitățile sunt separate în Pages, paginile principale care sunt asociate fiecărei rute în Router, Components, care conțin componente mai mici refolosite, ori de o singură dată în mai multe pagini, ori de o singură dată în mai multe pagini, și features, care conțin descrieri ale API-ului cu backend-ul (authApi.ts, postsApi.ts), descrieri de tipuri (authTypes.ts), sau componente logice (AuthSessionProvider.tsx).

În cookie-uri sunt stocate informații despre sesiunea curentă JWT, astfel încât utilizatorul poate să intre în platformă fără să trebuiască să se logeze de fiecare dată. Este reținută și scena la care lucra, astfel încât poată continua direct de unde s-a oprit.

3.2 Baza de date

Baza de date relațională este implementată în SQLite și este gestionată prin Entity Framework Core. Modelul ERD este compus din 8 tabele relaționale, conținând entitățile: Users (în implementare fiind AspNetUsers), GraphScenes, GraphSceneVersions, Posts, PostLikes, Comments, Reports, Bans. Există și entități create de modelul de Identity pentru stocarea și asocierea rolurilor utilizatorilor. Baza de date a fost proiectată pentru interogări eficiente, existând atribute care scot baza de date dintr-o formă normală pentru a optimiza interogările. Există relații one-to-many (o postare are mai multe comentarii) și many-to-many (postări și utilizatori care i-au dat like prin tabela PostLikes).

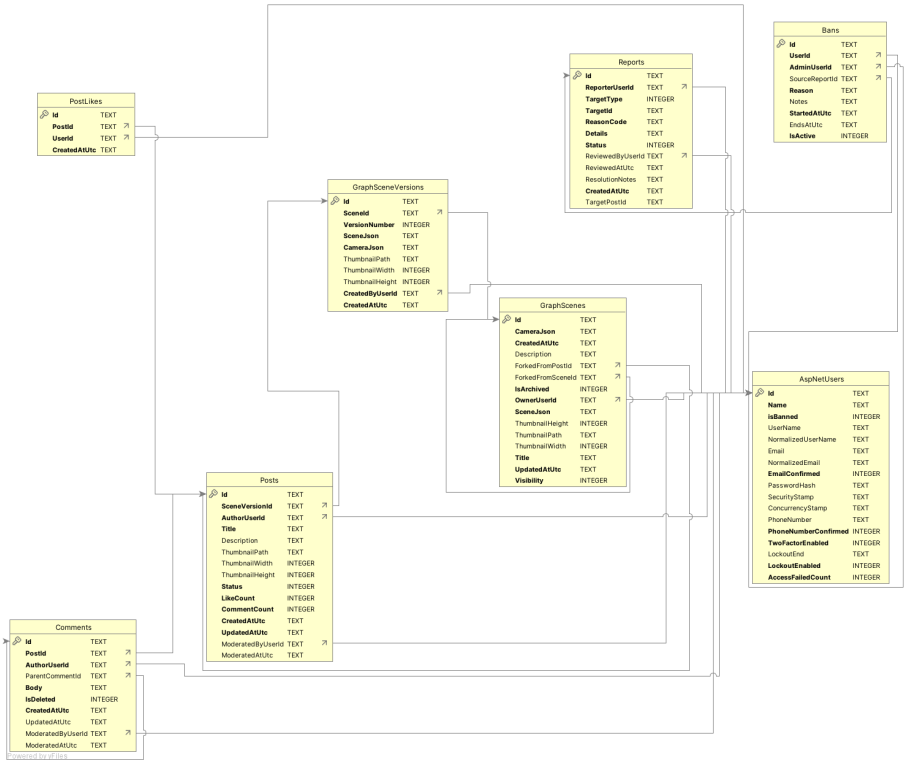


Figura 3.1: Arhitectura bazei de date.

Un utilizator poate să dețină multe scene de graf, să creeze mai multe versiuni de scene de graf, să creeze mai multe postări, să scrie mai multe comentarii. El poate să dea like la mai multe postări prin tabela asociativă PostLikes, să creeze mai multe raporturi, să analizeze mai multe raporturi, să emită mai multe ban-uri și să obțină mai multe ban-uri (ca utilizatorul banat).

Un GraphScene este deținut de un singur utilizator, și poate avea mai multe versiuni.

O versiune poate fi postată la maximum o postare.

O postare este deținută de un autor, și conține o versiune de scenă. Poate avea mai multe comentarii și mai multe like-uri. Opțional, poate fi și moderată de un admin.

Un comentariu are o singură postare și un singur autor.

Un like are o pereche unică de postare și autor.

Un raport este deținut de un utilizator raportant, un admin care s-a uitat peste el. Conține și o referință către fie postarea, fie comentariul raportat, tipul controlat de TargetType. Dacă e comentariu, conține și o referință către postarea părinte pentru acces rapid.

Un ban are un admin care a creat ban-ul și un utilizator care a fost banat. Un ban poate opțional să se refere și la un raport asociat. Utilizatorul are atributul isBanned, iar tabela Bans conține istoria de ban-uri.

3.3 Interfața de grafare

Interfața de grafare este centrul vizual și matematic al proiectului. Este implementată ca o componentă React, dar comportamentul ei este mai similar cu un engine mic de randare: acceptă expresii matematice simbolice, le compilează, ia sample-uri peste un domeniu, le transformă în geometrii Three.js și apoi le randează într-o scenă 3D live.

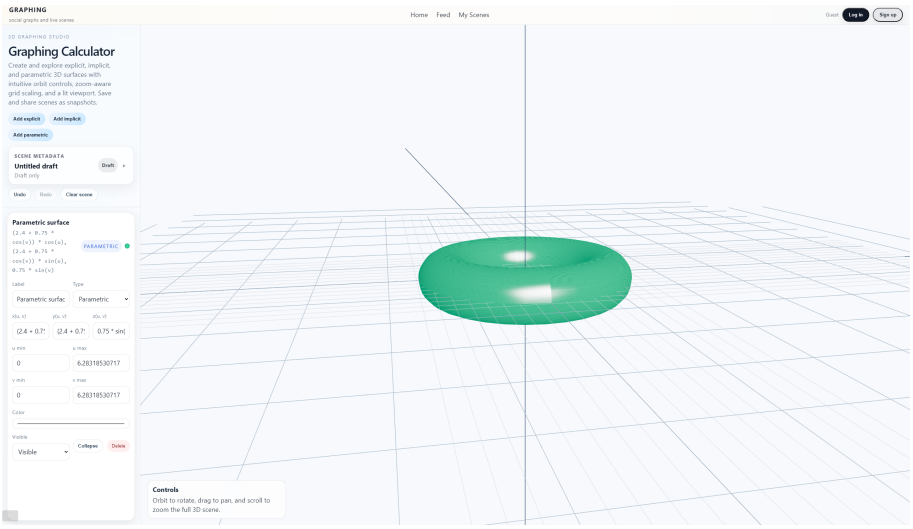


Figura 3.2: Interfața de grafare, cu un tor.

Sunt suportate trei tipuri de grafuri: explicite ($z = f(x, y)$), implicite ($F(x, y, z) = 0$) și parametrice ($(x(u, v), y(u, v), z(u, v))$). Ele sunt compilate folosind mathjs în expresii care vor fi folosite în generarea suprafețelor.

Suprafețele explicite sunt randate folosind THREE.PlaneGeometry. Expresiile parametrice, de asemenea, folosesc un plan, dar planul este folosit ca un domeniu pentru parametri, din care pozițiile 3D sunt alese. Expresiile implicite folosesc algoritmul *Marching Cubes*, explicat în 2.4.

Sistemul de grafare suportă logică de istorie a scenei, utilizatorul putând să dea undo și redo. Există suport și pentru salvarea și încărcarea de scene, cu metadate.

Capturi ale scenei sunt create când o scenă este salvată, folosind canvas.toDataURL("image/png"), care sunt trimise către server.

Interfața este una modernă și ușor de utilizat, cu dialoguri care apar când utilizatorul face o acțiune, și erori clare.

3.4 Roluri

Există două roluri principale: utilizatori normali și admini, împreună cu utilizatori neautentificați (guests). Fiecare utilizator are acces din ce în ce mai mare la platformă.

3.4.1 Utilizator neautentificat

Un utilizator neautentificat este unul care nu este logat în aplicație.

El are acces doar la pagina principală a aplicației, interfața de grafare, unde poate crea scene, însă nu le poate salva sau încărca altele. El, de asemenea, nu are acces la celelalte pagini, apărând un mesaj că nu are dreptul să acceseze pagina.

3.4.2 Utilizator normal

Utilizatorii normali sunt cei care sunt autentificați.

Un utilizator normal mai poate să salveze scene, și să acceseze paginile de My Scenes și Feed. În My Scenes se găsesc scenele pe care le-a creat, pe care poate să le șteargă, să le deschidă în editor, sau să creeze o postare cu ele. În Feed se găsesc postările făcute de utilizatori. Postările de la utilizatori banați nu vor apărea. Ei pot să reacționeze și să comenteze la ele, sau să deschidă o scenă (salvată) dintr-o postare. Ei mai pot și să dea raport fie la o postare, fie la un comentariu.

3.4.3 Admin

Adminii au rolul de a modera și de a menține controlul și buna funcționare a platformei, Ei sunt anumiți utilizatori care au rolul de 'admin' în baza de date.

Ei, pe lângă ce pot să facă utilizatorii normali, au acces la panoul pentru admini, unde pot vedea tote rapoartele și ban-urile create, putând să și acționeze peste ele. În Feed și paginile postărilor, ei mai pot și să șteargă postări sau comentarii sau să baneze utilizatori.

Capitolul 4

Ghidul de utilizare al aplicației

4.1 Utilizator neautentificat

Un utilizator neautentificat poate interacționa doar cu interfața de grafare și doar cu partea care nu are legătură cu interacționarea cu backend-ul. Astfel, singura pagină publică este cea a interfeței grafice.

Se pot ascunde anumite grafuri, și i se pot schimba culorile (4.1). Poate să dea zoom la scenă și să orbiteze camera în jurul centrului. Poate să șteargă scena curentă, apărând un mesaj (4.2).

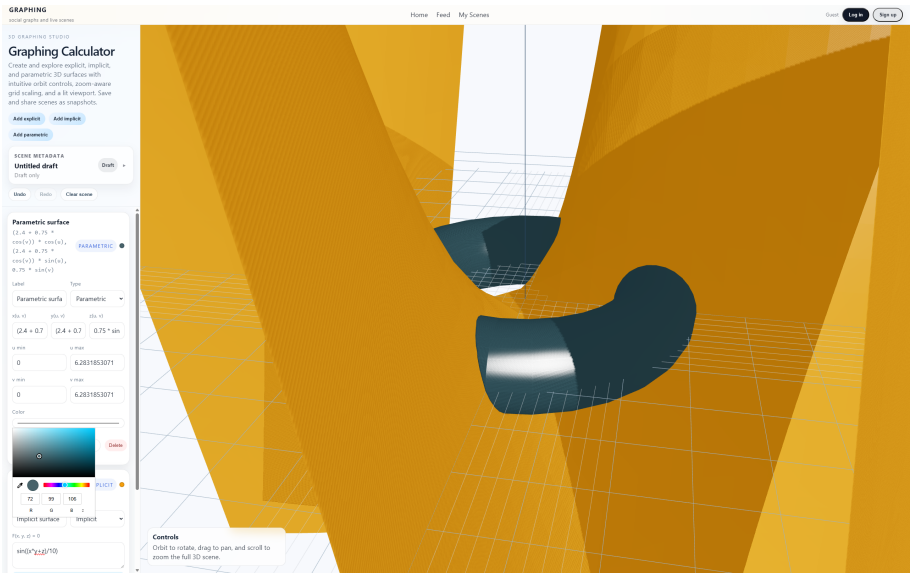


Figura 4.1: Interfața de grafare fiind folosită.

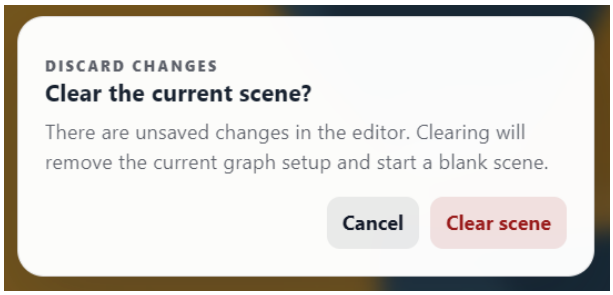


Figura 4.2: Mesaj de avertizare de ștergere a scenei curente.

Dacă încearcă să acceseze o pagină la care nu are acces, va apărea un mesaj că nu este logat (4.3).

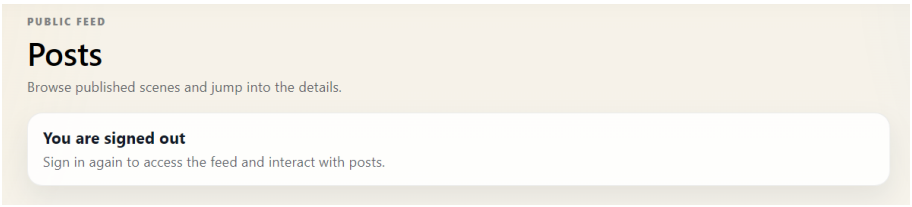
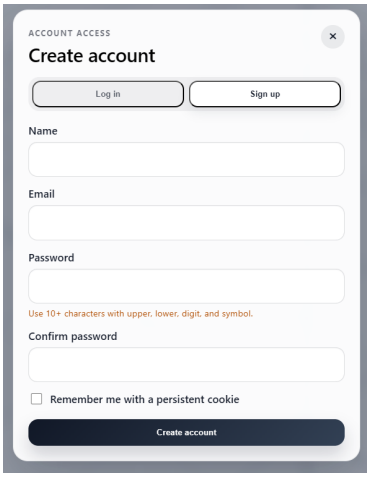
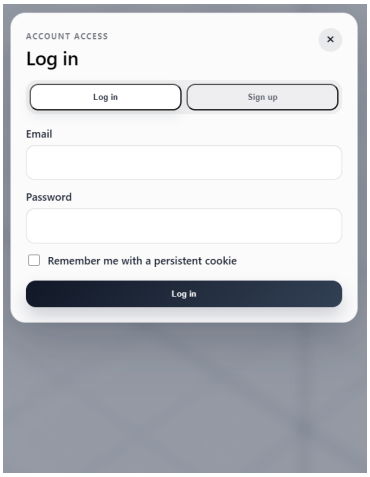


Figura 4.3: Mesaj de avertizare de accesare a unei pagini private.

Pagina de autentificare include opțiunea ca utilizatorul să fie amintit cu un cookie persistent. La înscriere, utilizatorul are și un câmp pentru nume, care va fi numele afișat public și trebuie să fie unic.



(a) Meniu de Sign Up.

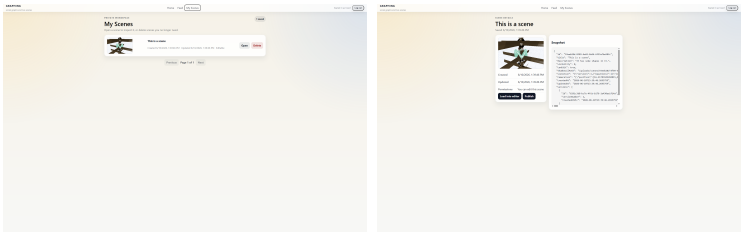


(b) Meniu de Log In.

4.2 Utilizator autentificat

După autentificare, utilizatorul are posibilitatea de a salva scene, care vor apărea pe pagina My Scenes (4.5). El poate să deschidă pagina pentru detaliile unei anumite scene, care va avea niște metadate, pe lângă forma JSON a scenei, și două butoane, unul

care deschide scena în interfața de grafare unde utilizatorul mai poate lucra și altul care deschide un formular pentru crearea unei postări (4.6).



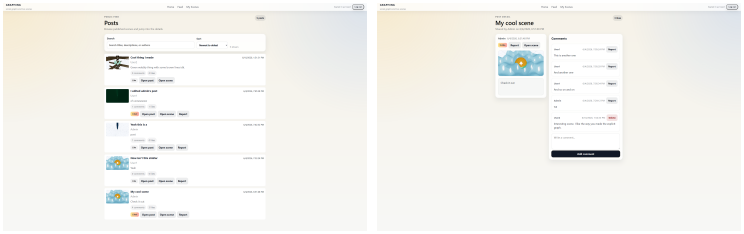
(a) Pagina de scene. (b) Pagina de detaliu pentru o anumită scenă.

Figura 4.5: Pagini aferente scenelor.



Figura 4.6: Formularul de postare.

El mai are acces și la funcționalitatea socială a platformei, aceasta fiind prezentă în Feed. Postările pot fi filtrate și pot fi sortate după dată și după popularitate. Poate să deschidă o postare să vadă comentariile și să posteze și el unele (4.7). El poate să acceseze scena ori prin apăsarea butoanelor relevante care sunt pe pagină ori prin apăsarea capturii de scenă. El poate raporta atât postări cât și comentarii create de alți utilizatori, având acces la un formular (4.8). Pot să aleagă diferite motive pentru raport.



(a) Feed-ul cu postări. (b) Pagina de detaliu pentru o postare.

Figura 4.7: Pagini aferente postărilor.

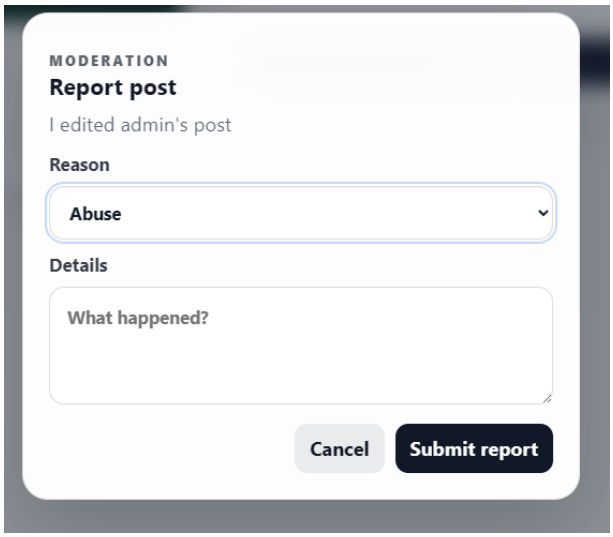
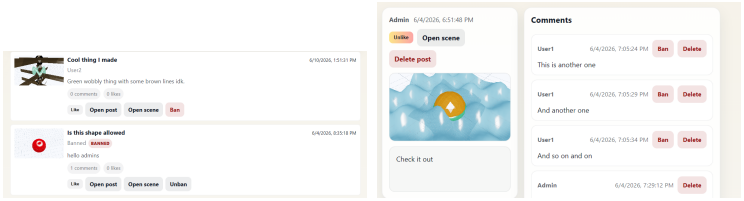


Figura 4.8: Formularul de raportare.

4.3 Admin

Adminii au acces la capabilități administrative și de moderare. Pot șterge postări și comentarii. Se observă că adminii nu mai pot raporta, opțiunea fiind înlocuită cu cea de-a bana utilizatorul. Postările banate vor apărea pentru administratori (4.9). Când apasă butonul de banare, apare un formular de banare, unde poate să introducă motivul (4.3).



(a) Feed-ul pentru un admin. (b) Postare pentru un admin.

Figura 4.9: Pagini aferente postărilor pentru un admin.

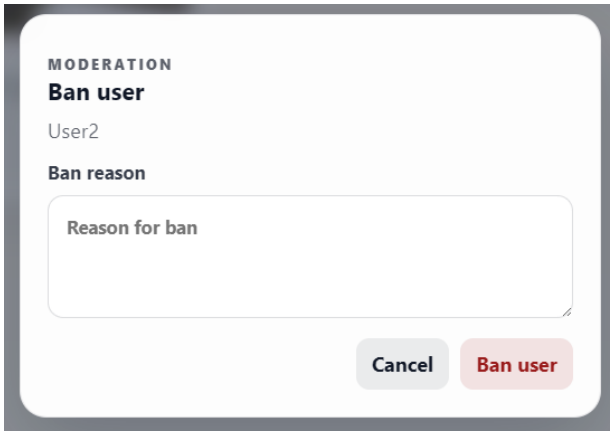


Figura 4.10: Formularul de raportare.

Ei pot accesa panelul pentru admini, care are linkuri către două subpagini, cea pentru raporturi și cea pentru banuri (4.11).

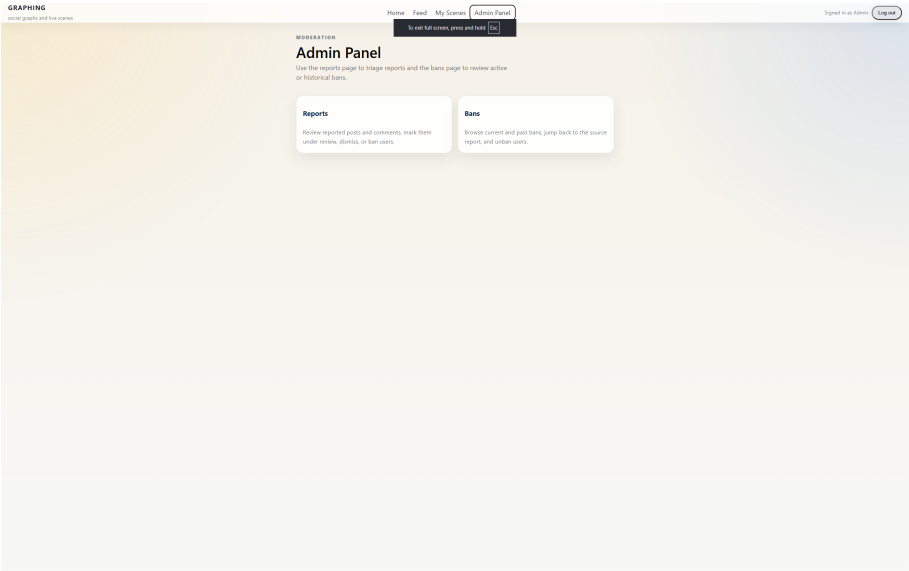


Figura 4.11: Panoul de administrație.

Pe pagina de raporturi, adminii pot vizualiza raporturile, atât cele vechi deja acționate, cât și cele active. Pot filtra după categoriile de rapoarte (open, under review, actioned, dismissed). După ce un raport este pus sub review, adminul poate bana direct utilizatorul din pagina de raporturi, ban-ul creat fiind asociat cu raportul. Poate și debana utilizatori care au fost banați în acest fel (4.12).

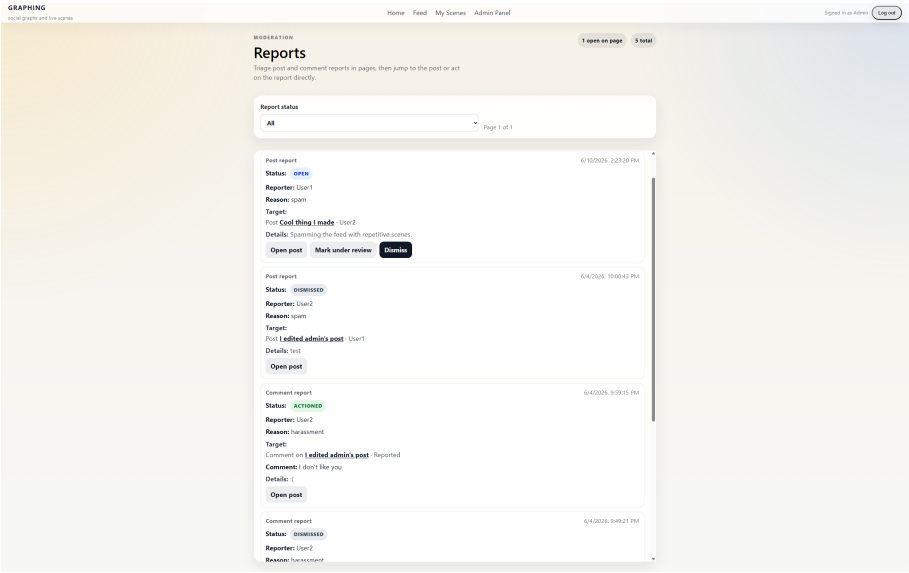


Figura 4.12: Pagina de raporturi.

Pe pagina de ban-uri, adminii pot vizualiza ban-urile, filtrând după toate sau doar după cele active. Pentru ban-urile active, adminii au opțiunea să debaneze utilizatorul

asociat ban-ului. Pentru fiecare ban apar metadate, precum și raportul asociat, dacă există (4.13).

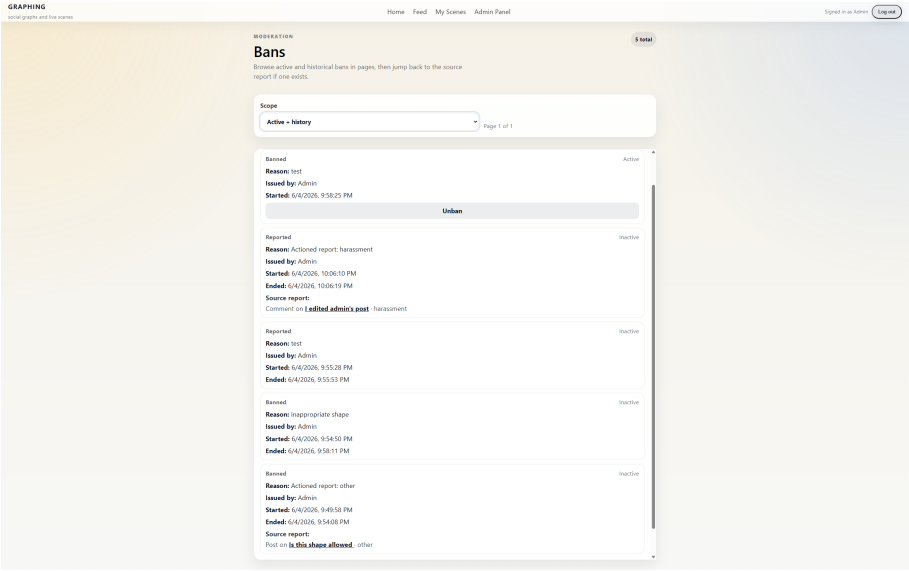


Figura 4.13: Pagina de raporturi.

Capitolul 5

Concluzii

Proiectul a avut ca scop dezvoltarea unei aplicații moderne, scalabile și securizate care îmbină elementele de grafare 3D cu elementele sociale. În comparație cu platformele sociale obișnuite, ale căror postări conțin text, imagini sau video-uri, unitatea principală de conținut folosită în acest caz sunt scenele de grafare interactive, randate într-un editor 3D în browser.

Pe parcursul lucrării s-au realizat analiza și proiectarea arhitecturii aplicației, implementând funcționalitățile esențiale într-o metodă eficientă, care duce la randarea rapidă a scenelor.

Aplicația a fost dezvoltată cu tehnologi moderne și rapide: .NET 8 pentru backend; React/Vite+TypeScript pentru frontend; autorizare și autentificare realizată prin JWT; baza de date SQLite; iar randarea scenelor prin Three.js împreună cu niște extensii, mathjs și isosurface.

Aplicația este una cu o interfață ușoară și intuitivă de folosit, cu un design responsive și curat. Rolurile sunt bine definite, iar drepturile și funcționalitățile lor sunt implementate clar și secur.

În concluzie, aplicația web implementată în acest proiect este una full-stack completă, folosind cele mai bune practici din domeniu. Ea constituie o bază solidă care poate fi extinsă mai departe.

În ceea ce privește metode specifice de îmbunătățire sau extindere a aplicației, există mai multe. O dată, sistemul grafic poate fi extins să ofere mai multe capabilități, precum: metode diferite de a randa o suprafață (transparență, hașurat, etc.); slidere pentru valori/suprafețe animate; suport nu doar pentru suprafețe, ci și pentru calcularea formulelor, cu funcții care pot fi definite; și îmbunătățirea și optimizarea în general a algoritmilor și a metodelor de randare folosite. În ceea ce privește platforma socială, pot fi adăugate: arbori de comentarii; editarea postărilor; adăugarea scenelor în comentarii, nu doar în postări; pagină de profil cu setări; abilitatea de a schimba parola și validarea email-urilor la înregistrare pe platformă. În final, backend-ul și, în mai specific, gestionarea capturilor de scenă ar putea fi îmbunătățite. Acuma, ele sunt salvate local pe server, însă ca să

servească un număr mare de utilizatori în viitor, ar trebui să se folosească un serviciu extern, precum Azure Blob Storage.

Bibliografie

[1] *ASP.NET Core Documentation*, Dec. 2025, URL: https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-8.0.

[2] *Entity Framework Core*, Dec. 2025, URL: https://learn.microsoft.com/en-us/ef/core/.

[3] *isosurface*, Mai 2026, URL: https://www.npmjs.com/package/isosurface.

[4] *JWT Introduction*, Ian. 2026, URL: https://www.jwt.io/introduction#when-to-use-json-web-tokens.

[5] *mathjs Documentation*, Mai 2026, URL: https://mathjs.org/docs/.

[6] *React*, Dec. 2025, URL: https://react.dev/.

[7] *SQLite*, Ian. 2026, URL: https://sqlite.org/.

[8] *Swagger Documentation*, Ian. 2026, URL: https://swagger.io/docs/.

[9] *Three.js Documentation*, Mar. 2026, URL: https://threejs.org/docs/.

[10] *Three.js Drei*, Mar. 2026, URL: https://drei.docs.pmnd.rs/getting-started/introductionn.

[11] *Three.js Fiber*, Mar. 2026, URL: https://r3f.docs.pmnd.rs/getting-started/introduction.

[12] *TypeScript Documentation*, Dec. 2025, URL: https://www.typescriptlang.org/docs/.

[13] *Vite*, Dec. 2025, URL: https://vite.dev/.